

BACKEND ENGINEERING

Complete Notes

From Fundamentals to Advanced Node.js Internals

Module 1 — Backend Fundamentals

Module 2 — JavaScript for Backend (with deep Event Loop coverage)

Module 3 — Node.js Complete

For Beginner to Advanced Backend Developers

Theory · Code Examples · Diagrams · Interview Q&A · Revision Notes

Table of Contents

Module 1 — Backend Fundamentals

- 1.1 What Is the Backend?
- 1.2 Frontend vs Backend
- 1.3 Client-Server Architecture
- 1.4 Request-Response Lifecycle
- 1.5 APIs (Application Programming Interfaces)
- 1.6 REST APIs
- 1.7 HTTP and HTTPS
- 1.8 URL Structure
- 1.9 HTTP Headers
- 1.10 Query Parameters
- 1.11 Request Body
- 1.12 HTTP Status Codes
- 1.13 Cookies
- 1.14 Sessions

Module 1 Revision Notes & Interview Questions

Module 2 — JavaScript for Backend

- 2.1 Variables: var, let, const
- 2.2 Functions
- 2.3 Objects
- 2.4 Arrays
- 2.5 Closures
- 2.6 Callbacks
- 2.7 Promises
- 2.8 Async/Await
- 2.9 The Event Loop — Deep Dive
- 2.10 Error Handling

Module 2 Revision Notes & Interview Questions

Module 3 — Node.js Complete

- 3.1 What Is Node.js?
- 3.2 The V8 Engine
- 3.3 Event-Driven Architecture
- 3.4 The Single-Threaded Model
- 3.5 Non-Blocking I/O
- 3.6 Core (Built-in) Modules — fs, path, http, os, events
- 3.7 npm (Node Package Manager)
- 3.8 package.json

Module 3 Revision Notes & Interview Questions

Module 1 — Backend Fundamentals

This module builds the mental model every backend engineer needs before touching a single line of framework code. We cover what "backend" actually means, how clients and servers talk to each other, and the building blocks of HTTP — the protocol that powers virtually all web communication.

1.1 What Is the Backend?

The backend is the part of an application that runs on a server, invisible to the end user, and is responsible for business logic, data storage, security, and serving the right data to the frontend on request. If the frontend is the restaurant's dining room — what the customer sees and interacts with — the backend is the kitchen: where ingredients (data) are stored, recipes (logic) are followed, and the final dish (response) is prepared before being sent out.

A typical backend system is responsible for:

- Handling HTTP requests and producing responses
- Implementing business rules (e.g. "a user cannot withdraw more than their balance")
- Talking to databases and caches to store and retrieve data
- Authenticating and authorizing users
- Integrating with third-party services (payment gateways, email providers, etc.)
- Ensuring data is processed securely, consistently, and efficiently

1.2 Frontend vs Backend

Frontend (also called "client-side") is everything the user sees and directly interacts with in their browser or app — buttons, forms, layouts, animations. Backend ("server-side") is everything that happens behind the scenes to make that interface actually work with real data.

Aspect	Frontend	Backend
Runs on	User's browser / device	A remote server
Languages	HTML, CSS, JavaScript (React, Vue, etc.)	JavaScript (Node.js), Python, Java, Go, Ruby, PHP, etc.
Responsible for	Layout, styling, user interaction, client-side state	Business logic, data storage, security, APIs
Visibility	Fully visible — user can inspect via DevTools	Hidden from the user entirely
Example task	Show a loading spinner while data loads	Query the database and return the data as JSON

1.3 Client-Server Architecture

Client-server architecture is a model in which one program (the client) requests resources or services, and another program (the server) fulfills those requests. The two communicate over a network, almost always using HTTP, and are usually on entirely different machines.

Key properties of this architecture:

- **Separation of concerns:** the client doesn't know how the server stores data, and the server doesn't know how the client renders the UI — they only agree on a contract (the API).
- **Many clients, one server (logically):** a single backend can serve millions of clients (web browsers, mobile apps, other servers) simultaneously.
- **Statelessness (in REST):** each request from the client typically contains all the information the server needs to process it — the server doesn't have to "remember" the client between requests.
- **Scalability:** because client and server are decoupled, you can scale them independently — add more servers without touching client code.

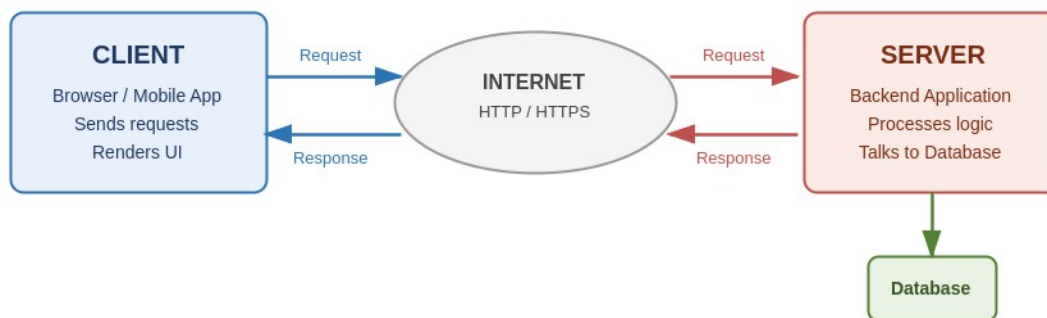


Figure 1.1 — Basic client-server architecture: requests flow to the server, responses flow back, and the server talks to a database.

1.4 Request-Response Lifecycle

Every single interaction on the web — clicking a link, submitting a form, an app fetching your feed — follows the same fundamental cycle. Understanding this lifecycle deeply is one of the highest-leverage things a backend engineer can learn, because almost every bug, slowdown, or security hole lives somewhere inside this cycle.

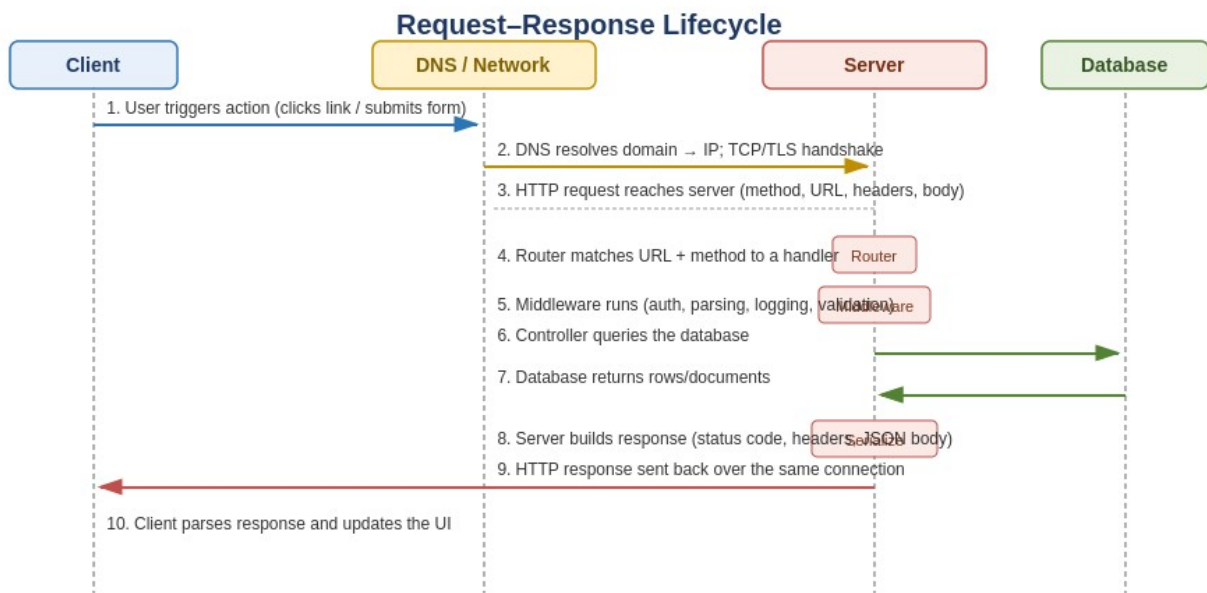


Figure 1.2 — The full request-response lifecycle from user action to UI update.

Lifecycle steps explained

- 1. User action:** the user clicks a button, submits a form, or the app fires a `fetch()` call.
- 2. DNS resolution:** the browser resolves the domain name (e.g. `api.example.com`) into an IP address.
- 3. Connection setup:** a TCP connection is opened, and if HTTPS, a TLS handshake encrypts the channel.
- 4. HTTP request sent:** the client sends a request containing a method, URL, headers, and optionally a body.
- 5. Routing:** the server's router inspects the method + URL path and decides which handler/controller should process it.
- 6. Middleware execution:** code runs before the main handler — e.g. authentication checks, request logging, body parsing, rate limiting.
- 7. Business logic / controller:** the actual logic runs — validating input, applying rules, calling services.
- 8. Database interaction:** the server reads or writes data to a database, cache, or external API as needed.
- 9. Response building:** the server constructs an HTTP response — a status code, headers, and a body (often JSON).
- 10. Response sent back:** the response travels back over the same connection to the client.
- 11. Client renders:** the client parses the response and updates the UI accordingly.

1.5 APIs (Application Programming Interfaces)

An API is a contract that defines how two pieces of software communicate — what requests are allowed, what data they expect, and what they will return. In a backend context, an API usually means a set of HTTP endpoints exposed by your server that a frontend (or another backend) can call.

Common types of APIs you'll encounter:

- **REST APIs:** resource-oriented, built on HTTP verbs and URLs (the most common style — see 1.6).
- **GraphQL APIs:** a single endpoint where clients specify exactly what data shape they want.
- **gRPC:** a high-performance, binary protocol often used for service-to-service communication.
- **WebSocket APIs:** persistent, two-way connections used for real-time features like chat or live notifications.

1.6 REST APIs

REST (Representational State Transfer) is an architectural style for designing networked applications. A REST API models everything as a "resource" (a user, an order, a product) and uses standard HTTP methods to perform operations on those resources, identified by URLs.

The 6 guiding principles of REST

- **Client-Server separation** — UI and data storage concerns are separated.
- **Statelessness** — no client session state is stored on the server between requests.
- **Cacheability** — responses must define themselves as cacheable or not, to improve performance.
- **Uniform interface** — resources are manipulated through a consistent, standard set of operations (HTTP verbs).
- **Layered system** — clients can't tell if they're talking directly to the server or through an intermediary (e.g. a load balancer).
- **Code on demand (optional)** — servers can extend client functionality by transferring executable code.

REST resource naming convention

Method	URL	Meaning
GET	/users	Get all users
GET	/users/42	Get the user with id 42
POST	/users	Create a new user
PUT	/users/42	Replace user 42 entirely
PATCH	/users/42	Partially update user 42

Method	URL	Meaning
DELETE	/users/42	Delete user 42

Best Practice

Use plural nouns for resource collections (/users, not /user), use nested paths for relationships (/users/42/orders), and never use verbs in REST URLs (avoid /getUser?id=42 — that's an RPC style, not REST).

1.7 HTTP and HTTPS

HTTP (HyperText Transfer Protocol) is the application-layer protocol that defines how clients and servers exchange messages. It is a request-response protocol: the client always initiates, and the server always responds.

HTTPS is HTTP layered on top of TLS (Transport Layer Security). It encrypts the data in transit so that an attacker intercepting the traffic cannot read or tamper with it. In modern backend development, HTTPS is mandatory — never HTTP — for anything handling real user data.

Common HTTP methods

Method	Purpose	Idempotent?	Safe?
GET	Retrieve data	Yes	Yes
POST	Create new data	No	No
PUT	Replace existing data entirely	Yes	No
PATCH	Partially update existing data	Not guaranteed	No
DELETE	Remove data	Yes	No
HEAD	Like GET but no response body	Yes	Yes
OPTIONS	Discover allowed methods/CORS preflight	Yes	Yes

- **Safe** = does not modify server state.
- **Idempotent** = calling it multiple times has the same effect as calling it once.

HTTP/1.1 vs HTTP/2 vs HTTP/3 (quick comparison)

Version	Format	Concurrency Model	Transport
HTTP/1.1	Text-based	One request per connection at a time (head-of-line blocking)	TCP

Version	Format	Concurrency Model	Transport
HTTP/2	Binary, multiplexed	Multiple requests in parallel over one connection	TCP
HTTP/3	Binary, multiplexed	Multiple independent streams, no head-of-line blocking at transport layer	QUIC (UDP-based)

1.8 URL Structure

A URL (Uniform Resource Locator) identifies where a resource lives and how to access it. Understanding each part precisely will save you from countless bugs around routing, query strings, and proxies.

https://api.example.com:443/v1/users/42?active=true&sort=name#bio						
scheme	host	port	path	query	fragment	

Part	Example	Description
Scheme	https	Protocol used (http, https, ftp, ws, wss)
Host	api.example.com	Domain name or IP address of the server
Port	443	TCP port the server listens on (443 default for HTTPS, 80 for HTTP)
Path	/v1/users/42	Identifies the specific resource on the server
Query string	active=true&sort=name	Key-value pairs providing extra parameters
Fragment	#bio	Client-side only — never sent to the server, used for in-page anchors

1.9 HTTP Headers

Headers are key-value pairs sent with every HTTP request and response that carry metadata — information about the message itself, rather than the actual content. They control caching, authentication, content negotiation, and more.

Common request headers

Header	Example Value	Purpose
Authorization	Bearer eyJhbGciOi...	Carries credentials (e.g. a JWT or API key)
Content-Type	application/json	Tells the server the format of the request body
Accept	application/json	Tells the server what response formats the

Header	Example Value	Purpose
		client can handle
User-Agent	Mozilla/5.0...	Identifies the client application/browser
Cookie	sessionId=abc123	Sends stored cookies back to the server

Common response headers

Header	Example Value	Purpose
Content-Type	application/json; charset=utf-8	Format of the response body
Set-Cookie	sessionId=abc123; HttpOnly	Instructs the client to store a cookie
Cache-Control	no-cache, max-age=3600	Caching rules for the response
Access-Control-Allow-Origin	*	CORS — which origins may read this response

1.10 Query Parameters

Query parameters are key-value pairs appended to a URL after a ? symbol, used to filter, sort, paginate, or otherwise modify a request — without changing which resource (path) is being targeted.

```
GET /products?category=shoes&minPrice=20&maxPrice=100&page=2
```

Reading query parameters in an Express (Node.js) route handler:

```
app.get('/products', (req, res) => {
  const { category, minPrice, maxPrice, page } = req.query;
  // category = 'shoes', minPrice = '20' (always a string!)
  res.json({ category, minPrice, maxPrice, page });
});
```

Gotcha

All query parameter values arrive as strings, even if they look like numbers or booleans. Always parse/validate them (`Number(minPrice)`, or a schema validator) before using them in logic or database queries.

Use query params for filtering, sorting, and pagination on GET requests. Use the URL path for identifying a specific resource (e.g. `/users/42`, not `/users?id=42`).

1.11 Request Body

The body carries the actual data payload of a request — typically used with POST, PUT, and PATCH to send data the server should create or update. GET requests should not carry a body.

The most common body format in modern APIs is JSON:

```
POST /users HTTP/1.1
Content-Type: application/json

{
  "name": "Aanya Sharma",
  "email": "aanya@example.com",
  "age": 28
}
```

On the server (Express), you need body-parsing middleware to read it:

```
const express = require('express');
const app = express();

app.use(express.json()); // parses Content-Type: application/json

app.post('/users', (req, res) => {
  const { name, email, age } = req.body;
  res.status(201).json({ message: 'User created', name, email, age });
});
```

1.12 HTTP Status Codes

Status codes are 3-digit numbers in the response that tell the client what happened to its request, grouped into five classes.

Class	Name	Meaning	Examples
1xx	Informational	Request received, continuing process	100 Continue
2xx	Success	The request was successfully received and processed	200 OK, 201 Created, 204 No Content
3xx	Redirection	Further action is needed to complete the request	301 Moved Permanently, 304 Not Modified
4xx	Client Error	The client sent a bad or invalid request	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Error	The server failed to fulfil a valid request	500 Internal Server Error, 503 Service Unavailable

Status codes every backend engineer must know cold

Code	Name	When to Use
200	OK	Standard success response
201	Created	A new resource was successfully created
204	No Content	Success, but there's nothing to return (e.g. after DELETE)
400	Bad Request	Malformed request syntax or invalid data
401	Unauthorized	Authentication is missing or invalid
403	Forbidden	Authenticated, but not allowed to access this resource
404	Not Found	Resource does not exist
409	Conflict	Request conflicts with current server state (e.g. duplicate email)
422	Unprocessable Entity	Syntax is fine, but semantic validation failed
429	Too Many Requests	Client has been rate-limited
500	Internal Server Error	Unhandled exception on the server
503	Service Unavailable	Server temporarily can't handle the request (overload/maintenance)

1.13 Cookies

A cookie is a small piece of data that the server asks the browser to store, and which the browser automatically sends back on every subsequent request to that domain. Cookies are the original mechanism for adding state on top of stateless HTTP.

```
// Server sets a cookie
res.cookie('sessionId', 'abc123', {
  httpOnly: true, // JavaScript on the page cannot read it (XSS protection)
  secure: true,   // only sent over HTTPS
  sameSite: 'strict', // not sent on cross-site requests (CSRF protection)
  maxAge: 24 * 60 * 60 * 1000, // 1 day in ms
});
```

Key cookie attributes

Attribute	Purpose
HttpOnly	Prevents client-side JavaScript from accessing the cookie (mitigates XSS theft)

Attribute	Purpose
Secure	Cookie is only sent over HTTPS connections
SameSite	Controls whether the cookie is sent with cross-site requests (Strict / Lax / None)
Expires / Max-Age	Determines how long the cookie persists before the browser deletes it
Domain / Path	Scopes which URLs the cookie is sent to

1.14 Sessions

A session is server-side storage that tracks a user's state across multiple requests. Because HTTP is stateless by nature, sessions are how applications "remember" that you're logged in as you navigate from page to page.

How session-based authentication works

1. User logs in with credentials (username/password).
2. Server verifies credentials, then creates a session record (often in memory, Redis, or a database) with a unique session ID.
3. Server sends that session ID back to the client as a cookie (e.g. Set-Cookie: sessionId=abc123).
4. On every future request, the browser automatically includes that cookie.
5. Server looks up the session ID in its store, retrieves the associated user data, and treats the request as authenticated.

Sessions vs Tokens (JWT) — a common interview topic

Aspect	Session-based Auth	Token-based Auth (JWT)
Storage location	Server-side (memory/Redis/DB)	Client-side (in the token itself)
Server memory needed	Yes — one record per active user	No — server is stateless, just verifies a signature
Revocation	Easy — delete the session record	Hard — token is valid until it expires unless you track a blocklist
Scalability across servers	Needs shared session store	Naturally stateless and scalable
Typical use case	Traditional web apps	APIs, mobile apps, microservices

Module 1 — Revision Notes (Quick Recall)

- Backend = server-side logic, data, and security; Frontend = client-side UI.
- Client-server model: client requests, server responds; decoupled and scalable.

- Request-response lifecycle: DNS → TCP/TLS → request → routing → middleware → controller → DB → response → render.
- REST = resource-based design using standard HTTP verbs and URLs (no verbs in URLs).
- HTTPS = HTTP + TLS encryption; always required in production.
- URL parts: scheme, host, port, path, query string, fragment (fragment never sent to server).
- Headers carry metadata (auth, content type, caching); body carries the data payload.
- Query params: always strings; used for filtering/sorting/pagination on GET.
- Status codes: 2xx success, 4xx client error, 5xx server error — know 200/201/204/400/401/403/404/409/422/429/500 cold.
- Cookies: small client-stored data, auto-sent with requests; secure with HttpOnly, Secure, SameSite.
- Sessions: server-side state keyed by a session ID stored in a cookie; JWTs are the stateless alternative.

Module 1 — Interview Questions

Q1. What is the difference between frontend and backend?

A: Frontend is the client-side layer the user directly sees and interacts with (HTML/CSS/JS rendered in a browser or app). Backend is the server-side layer handling business logic, data persistence, authentication, and serving data to the frontend through APIs. The user never directly sees backend code.

Q2. Explain the request-response cycle in your own words.

A: A client sends an HTTP request containing a method, URL, headers, and optionally a body. The server's router matches it to a handler, middleware runs (auth, parsing, logging), the controller executes business logic and talks to the database if needed, then the server builds and returns a response with a status code, headers, and a body. The client then parses that response and updates its UI.

Q3. What makes an API "RESTful"?

A: It follows REST's architectural constraints: client-server separation, statelessness, cacheability, a uniform interface (standard HTTP verbs operating on resource URLs), and a layered system. In practice this means resources are nouns in the URL (/orders/5) and actions are expressed through HTTP methods (GET, POST, PUT, PATCH, DELETE), not verbs in the URL.

Q4. What is the difference between PUT and PATCH?

A: PUT replaces the entire resource with the data provided — any fields omitted are typically wiped or reset. PATCH applies a partial update, modifying only the fields included in the request body, leaving the rest untouched.

Q5. Why is HTTP described as 'stateless'?

A: Because the protocol itself does not retain any memory of previous requests — every request is treated independently, with no built-in concept of a 'logged in' user between calls. That's exactly why cookies and sessions (or tokens) had to be invented on top of HTTP to simulate continuity.

Q6. What's the difference between 401 and 403 status codes?

A: 401 Unauthorized means the request lacks valid authentication credentials — the server doesn't know who you are (or your credentials are invalid/expired). 403 Forbidden means the server knows who you are, but you don't have permission to access that specific resource.

Q7. How do sessions differ from JWTs for authentication?

A: Sessions store state on the server (keyed by a session ID sent via cookie), which makes revocation trivial but requires shared storage when scaling across multiple servers. JWTs store the state inside a signed token on the client, making the server stateless and easy to scale horizontally, but revoking a single token before it expires is harder, usually needing a blocklist.

Q8. Why should query parameters never be trusted as their literal types?

A: Because the query string is just text in a URL — every value arrives as a string regardless of how it looks (e.g. "42" not 42, "true" not true). Backend code must explicitly parse and validate them before use, otherwise type-related bugs and injection vulnerabilities can creep in.

Q9. What's the role of the HttpOnly and Secure flags on a cookie?

A: HttpOnly prevents JavaScript running on the page from reading the cookie via `document.cookie`, which blocks a major class of XSS-based session theft. Secure ensures the cookie is only ever transmitted over HTTPS, preventing it from being sniffed on an unencrypted connection.

Q10. What is idempotency, and which HTTP methods are idempotent?

A: An idempotent operation produces the same end state no matter how many times it's executed. GET, PUT, DELETE, and HEAD are idempotent — calling DELETE `/users/5` five times still results in user 5 being gone, with no compounding side-effects. POST is not idempotent, since calling it repeatedly typically creates multiple new resources.

Module 2 — JavaScript for Backend

Node.js runs JavaScript, so a backend engineer needs a rock-solid grasp of the language itself — not just syntax, but how it actually executes under the hood, especially around asynchronous code. This module covers the core language features and then goes deep on the single most misunderstood topic for backend developers: the Event Loop.

2.1 Variables: var, let, const

JavaScript has three ways to declare variables, each with different scoping and mutability rules.

Keyword	Scope	Re-assignable	Hoisted & usable before declaration
var	Function-scoped	Yes	Yes (initialized as undefined)
let	Block-scoped	Yes	No (ReferenceError if accessed early)
const	Block-scoped	No (reference is fixed)	No (ReferenceError if accessed early)

```
function demo() {
  if (true) {
    var x = 'var value';
    let y = 'let value';
  }
  console.log(x); // 'var value' -> leaked outside the block
  console.log(y); // ReferenceError -> y is block-scoped
}

const user = { name: 'Riya' };
user.name = 'Aman'; // OK -> mutating the object is allowed
user = {};          // TypeError -> re-assigning the binding is NOT allowed
```

Best Practice

Default to const. Use let only when you know the variable will be reassigned (loop counters, accumulators). Avoid var entirely in modern backend code — its function-scoping causes subtle bugs.

2.2 Functions

Functions are first-class citizens in JavaScript — they can be stored in variables, passed as arguments, and returned from other functions. This is the foundation that makes callbacks, promises, and middleware patterns possible.

```
// Function declaration -> hoisted, can be called before its definition
function add(a, b) {
  return a + b;
}

// Function expression -> NOT hoisted the same way
const subtract = function (a, b) {
  return a - b;
};

// Arrow function -> shorter syntax, no own 'this' binding
const multiply = (a, b) => a * b;

// Default + rest parameters
function createUser(name, role = 'guest', ...permissions) {
  return { name, role, permissions };
}
```

Arrow functions vs regular functions

Feature	Regular Function	Arrow Function
this binding	Own dynamic this	Inherits this from enclosing scope (lexical)
Can be a constructor	Yes (new Foo())	No
arguments object	Has its own	Does not have its own
Best for	Object methods, constructors	Callbacks, short pure functions

2.3 Objects

Objects are JavaScript's core structure for representing structured data — exactly what you'll be sending and receiving as JSON in nearly every backend route.

```
const product = {
  id: 101,
  name: 'Mechanical Keyboard',
  price: 79.99,
  inStock: true,
  getDiscountPrice(percent) { // method shorthand
    return this.price * (1 - percent / 100);
  },
};
```

```
// Destructuring -> extremely common when reading req.body
const { name, price } = product;

// Spread -> shallow copy / merging
const updated = { ...product, price: 69.99 };

// Optional chaining + nullish coalescing -> safe access
const city = product?.address?.city ?? 'Unknown';
```

2.4 Arrays

Arrays are ordered collections, and their higher-order methods (map, filter, reduce) are used constantly when transforming database results before sending a response.

```
const orders = [
  { id: 1, total: 250, status: 'paid' },
  { id: 2, total: 80, status: 'pending' },
  { id: 3, total: 430, status: 'paid' },
];

const paidOrders = orders.filter(o => o.status === 'paid');
const totals = orders.map(o => o.total);
const revenue = orders.reduce((sum, o) => sum + o.total, 0);
const hasPending = orders.some(o => o.status === 'pending');
const allPaid = orders.every(o => o.status === 'paid');
const found = orders.find(o => o.id === 2);
```

Method	Purpose	Returns
map	Transform each element	New array, same length
filter	Keep elements matching a condition	New array, same or fewer elements
reduce	Accumulate into a single value	Any single value (number, object, etc.)
find / findIndex	Get the first match	Single element / index, or undefined/-1
some / every	Test elements against a condition	Boolean

2.5 Closures

A closure is formed when a function "remembers" the variables from the scope in which it was created, even after that outer scope has finished executing. Closures are the mechanism behind private state, memoization, and many middleware/factory patterns in backend code.

```
function createRateLimiter(limit) {
  let count = 0; // private state, captured by closure

  return function (req, res, next) {
    count++;
    if (count > limit) {
      return res.status(429).json({ error: 'Too many requests' });
    }
    next();
  };
}

const limiter = createRateLimiter(100); // 'count' and 'limit' live on
app.use(limiter); // inside this closure forever
```

Here, `count` and `limit` are not global variables — they're trapped inside the closure returned by `createRateLimiter`. Every call to the returned middleware function shares and updates that same private `count`, but nothing outside the closure can directly touch it.

2.6 Callbacks

A callback is simply a function passed as an argument to another function, to be "called back" later — typically once some operation (often asynchronous) completes. Callbacks were JavaScript's original mechanism for handling async work, before Promises existed.

```
const fs = require('fs');

fs.readFile('data.json', 'utf-8', (err, data) => {
  if (err) {
    console.error('Failed to read file:', err);
    return;
  }
  console.log('File contents:', data);
});

console.log('This logs BEFORE the file content!'); // proves it's async
```

The callback hell problem

When multiple async operations depend on each other, nesting callbacks creates deeply indented, hard-to-read code often called "callback hell" or the "pyramid of doom":

```
getUser(userId, (err, user) => {
  if (err) return handleError(err);
  getOrders(user.id, (err, orders) => {
    if (err) return handleError(err);
    getInvoice(orders[0].id, (err, invoice) => {
      if (err) return handleError(err);
```

```

    console.log(invoice); // 4 levels deep and still growing
  });
});
});

```

Why this matters

Callback hell is exactly the problem Promises and `async/await` were invented to solve — you'll see the same logic rewritten cleanly later in this module.

2.7 Promises

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation. It is JavaScript's structured replacement for nested callbacks, and it exists in exactly one of three states at any time.

State	Meaning
Pending	Initial state — operation not yet complete
Fulfilled	Operation completed successfully — <code>.then()</code> handlers run
Rejected	Operation failed — <code>.catch()</code> handlers run

```

function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id <= 0) {
        reject(new Error('Invalid user id'));
      } else {
        resolve({ id, name: 'Karan Mehta' });
      }
    }, 1000);
  });
}

fetchUser(5)
  .then(user => console.log('User:', user))
  .catch(err => console.error('Error:', err.message))
  .finally(() => console.log('Request finished'));

```

Promise combinators

Method	Behavior
<code>Promise.all([...])</code>	Waits for ALL to resolve; rejects immediately if ANY rejects
<code>Promise.allSettled([...])</code>	Waits for ALL to finish (resolve or reject); never short-circuits

Method	Behavior
Promise.race([...])	Resolves/rejects as soon as the FIRST promise settles
Promise.any([...])	Resolves as soon as the FIRST one fulfills; rejects only if ALL reject

```
const results = await Promise.all([
  fetchUser(1),
  fetchOrders(1),
  fetchInvoices(1),
]); // runs all three concurrently, not one after another
```

2.8 Async/Await

async/await is syntactic sugar built on top of Promises that lets asynchronous code read like synchronous code, dramatically improving readability. An async function always returns a Promise, and await pauses execution of that function (not the whole program) until the awaited Promise settles.

The earlier callback-hell example, rewritten cleanly:

```
async function getInvoiceForFirstOrder(userId) {
  try {
    const user = await getUser(userId);
    const orders = await getOrders(user.id);
    const invoice = await getInvoice(orders[0].id);
    return invoice;
  } catch (err) {
    handleError(err);
  }
}
```

Critical Detail

await only pauses the function it's inside — it does NOT block the rest of your application. Other requests, timers, and events continue to be processed by the event loop while this function is 'paused' waiting on the Promise.

Sequential vs concurrent awaits — a common performance bug

```
// SLOW -> sequential: each await waits before starting the next (3s total)
const a = await taskThatTakes1Second();
const b = await taskThatTakes1Second();
const c = await taskThatTakes1Second();

// FAST -> concurrent: all three start immediately (1s total)
const [a, b, c] = await Promise.all([
  taskThatTakes1Second(),
```

```
taskThatTakes1Second(),  
taskThatTakes1Second(),  
]);
```

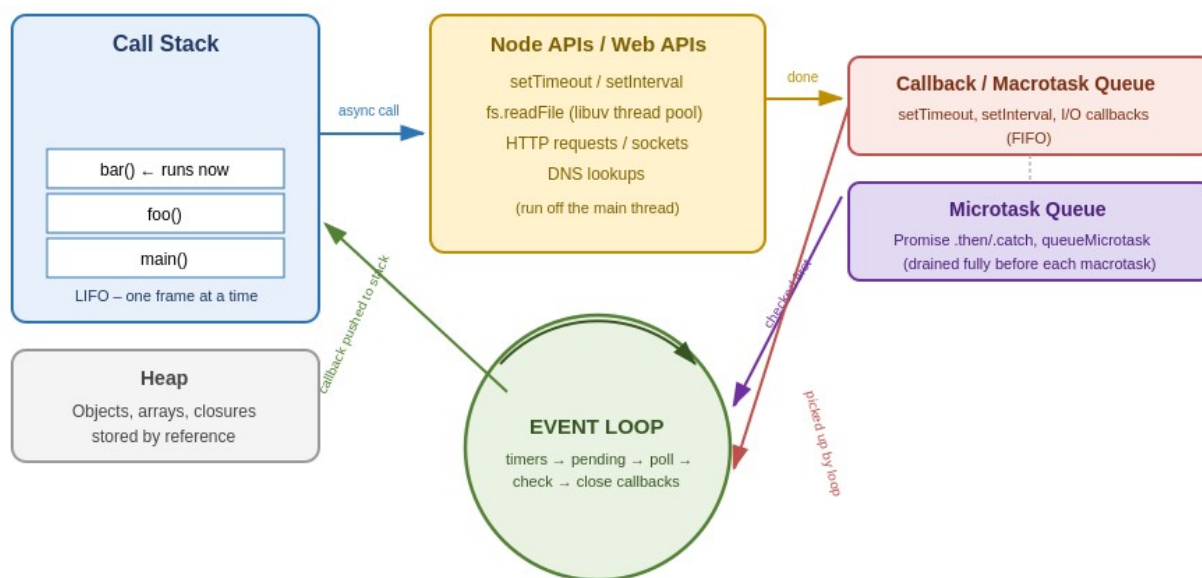
2.9 The Event Loop — Deep Dive

This is the single most important concept for understanding why JavaScript behaves the way it does, and it comes up in nearly every backend interview. JavaScript is single-threaded — it can only execute one piece of code at a time — yet it handles thousands of concurrent operations (database calls, file reads, network requests) without ever "freezing." The Event Loop is the mechanism that makes this possible.

The building blocks

- **Call Stack** — where JavaScript tracks what function is currently executing. It's Last-In-First-Out (LIFO): the most recently called function finishes first.
- **Heap** — where objects, arrays, and closures are actually stored in memory.
- **Web APIs / Node APIs** — browser or Node-provided functionality (timers, file I/O, network calls) that runs OUTSIDE the main JS thread.
- **Callback / Macrotask Queue** — holds callbacks from `setTimeout`, `setInterval`, and I/O, waiting to be run, in FIFO order.
- **Microtask Queue** — holds `Promise .then/.catch/.finally` callbacks and `queueMicrotask()` callbacks. Always drained completely before the next macrotask.
- **Event Loop itself** — a continuously running process that checks: "Is the call stack empty? If so, what's next in the queues?"

JavaScript Runtime: Call Stack, Event Loop & Queues



Rule: stack must be EMPTY before the event loop pushes any callback onto it.
 Rule: ALL microtasks drain completely before the next macrotask runs.

Figure 2.1 — The full picture: Call Stack, Heap, Web/Node APIs, Macrotask Queue, and Microtask Queue, all coordinated by the Event Loop.

The golden rule

Memorize this

The event loop always fully empties the microtask queue before it processes even ONE macrotask. Within macrotasks, only one is processed per loop tick, but ALL pending microtasks are drained after every single callback — whether it's a macrotask or another microtask.

Step-by-step trace of a real example

Let's trace through this snippet line by line to see exactly why the output order is what it is — this exact kind of question appears constantly in interviews:

Execution Trace: Sync vs setTimeout vs Promise

```
console.log('A');
setTimeout (() => {
  console.log('B');
}, 0);
Promise.resolve().then(() => {
  console.log('C');
});
console.log('D');
```

Console Output (in order):

```
A
D
C ← microtask runs before timer
B ← macrotask runs last
```

Step-by-step:

- 1 console.log('A') runs immediately on the call stack → prints A
- 2 setTimeout registers callback with Web/Node API; timer starts (0ms); main thread moves on
- 3 Promise.resolve().then() schedules its callback in the MICROTASK queue, not the stack
- 4 console.log('D') runs immediately → prints D
- 5 Call stack is now EMPTY. Engine checks microtask queue first – always, every time
- 6 Microtask queue has the .then callback → runs it fully → prints C
- 7 Microtask queue is now empty. Event loop moves to the timers phase (macrotasks)
- 8 Timer's 0ms has elapsed → its callback is pushed to the call stack → prints B

Golden Rule: ALL synchronous code → ALL microtasks → ONE macrotask → repeat

Figure 2.2 — Execution trace showing exactly why synchronous code, microtasks, and macrotasks print in the order A, D, C, B.

The takeaway: synchronous code always runs first and completely, in order. Then ALL queued microtasks run, fully draining the queue — even if new microtasks get added while draining. Only after the microtask queue is completely empty does the event loop move on to a single macrotask.

Node.js specifics: the libuv phases

In the browser, the event loop concept is simpler. In Node.js, the event loop (implemented by a C library called libuv) is broken into distinct phases, each with its own queue, executed in a fixed order on every "tick" of the loop.

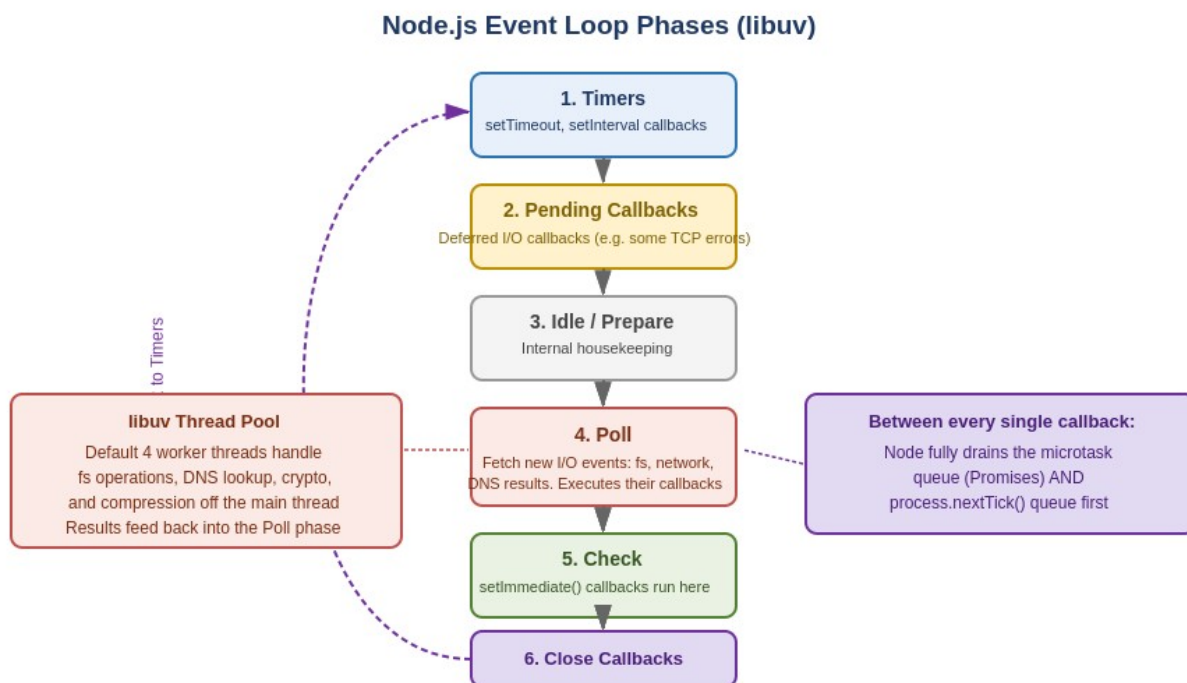


Figure 2.3 — The six phases of Node.js's libuv event loop, plus where the thread pool and microtasks fit in.

Phase	What Happens
Timers	Executes callbacks scheduled by <code>setTimeout()</code> and <code>setInterval()</code> whose timer has expired
Pending Callbacks	Executes some I/O callbacks deferred from the previous loop iteration (e.g. certain TCP errors)
Idle / Prepare	Internal libuv bookkeeping, rarely used directly by application code
Poll	The heart of the loop — retrieves new I/O events (file reads, network data) and runs their callbacks; the loop may block here waiting for events
Check	Executes <code>setImmediate()</code> callbacks — designed to run right after the Poll phase
Close Callbacks	Executes 'close' event callbacks, e.g. <code>socket.on('close', ...)</code> , before starting the next loop

setTimeout vs setImmediate vs process.nextTick

This is a classic Node.js-specific interview trap because the three behave very differently in execution priority:

API	Priority	When It Actually Runs
<code>process.nextTick()</code>	Highest — runs before any microtask or phase	Runs immediately after the current operation, before the event loop continues

API	Priority	When It Actually Runs
Promise microtasks	Second — after nextTick queue, before any macrotask phase	Runs after the current operation, after nextTick queue drains
setImmediate()	Runs in the Check phase	Designed to run once per loop iteration, after I/O events in Poll
setTimeout(fn, 0)	Runs in the Timers phase	Runs after at least the specified delay (0ms is a minimum, not a guarantee)

```

console.log('start');

setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
process.nextTick(() => console.log('nextTick'));
Promise.resolve().then(() => console.log('promise'));

console.log('end');

// Output:
// start
// end
// nextTick    <- always first among the async items
// promise     <- microtask, drains right after nextTick queue
// timeout / immediate <- order between these two can vary!

```

Interview Trap

The relative order of `setTimeout(fn, 0)` and `setImmediate()` is NOT guaranteed when called from the main module — it depends on timing precision and system load. But inside an I/O callback (e.g. within `fs.readFile`'s callback), `setImmediate()` is ALWAYS guaranteed to run before `setTimeout(fn, 0)`, because Check runs right after Poll, before the loop reaches Timers again.

Why the event loop matters for backend performance

Node.js's entire performance model depends on never blocking the event loop. A single slow, synchronous operation (a huge `JSON.parse`, a CPU-heavy loop, a synchronous file read) freezes the ENTIRE server — every other request, regardless of which user it belongs to, has to wait, because there's only one thread running your JS.

```

// BAD -> blocks the event loop, freezing ALL other requests
app.get('/report', (req, res) => {
  const data = fs.readFileSync('huge-file.csv'); // synchronous!
  res.send(processSync(data));                    // CPU-heavy!
});

// GOOD -> non-blocking I/O frees the event loop for other requests
app.get('/report', async (req, res) => {

```

```
const data = await fs.promises.readFile('huge-file.csv');
res.send(await processAsync(data));
});
```

2.10 Error Handling

Robust backend code anticipates failure — network calls fail, databases time out, users send invalid data. JavaScript gives you several layers of error handling, each suited to a different style of code.

try/catch with synchronous and async code

```
// Synchronous
try {
  const parsed = JSON.parse(invalidJsonString);
} catch (err) {
  console.error('Parsing failed:', err.message);
}

// Async/await -> try/catch works naturally here
async function getUser(id) {
  try {
    const user = await db.users.findById(id);
    if (!user) throw new Error('User not found');
    return user;
  } catch (err) {
    logger.error(err);
    throw err; // re-throw so the caller can also react
  }
}
```

Custom error classes

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}

class NotFoundError extends AppError {
  constructor(message = 'Resource not found') {
    super(message, 404);
  }
}

// usage in a controller
if (!user) throw new NotFoundError('User not found');
```

Centralized Express error-handling middleware

```
// Must be defined LAST, after all routes, with 4 arguments
app.use((err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    error: err.message || 'Internal Server Error',
  });
});
```

Catching unhandled rejections and exceptions globally

```
process.on('unhandledRejection', (reason) => {
  logger.error('Unhandled Rejection:', reason);
  // gracefully shut down -> the app may be in an unknown state
  process.exit(1);
});

process.on('uncaughtException', (err) => {
  logger.error('Uncaught Exception:', err);
  process.exit(1);
});
```

Best Practice

Never silently swallow errors with an empty catch block. Always log, and either handle meaningfully or re-throw. Distinguish 'operational errors' (expected, e.g. invalid input) from 'programmer errors' (bugs) — the former should produce clean API responses, the latter should be logged loudly and may warrant a process restart.

Module 2 — Revision Notes (Quick Recall)

- `const` by default, `let` when reassignment is needed, avoid `var`.
- Arrow functions inherit `'this'` lexically and cannot be used as constructors.
- Destructuring + spread are used everywhere when working with `req.body` / `req.query`.
- Array methods: `map` (transform), `filter` (select), `reduce` (accumulate), `find/some/every` (search/test).
- Closures let functions retain private state — the backbone of middleware factories.
- Callbacks → Promises → `async/await` is an evolution to solve nested 'callback hell.'
- Promise states: pending, fulfilled, rejected. Use `Promise.all` for concurrent independent tasks.
- `await` pauses only the current async function, never blocks the whole runtime.
- Event Loop golden rule: all sync code → drain ALL microtasks → ONE macrotask → repeat.
- Node's libuv phases in order: Timers → Pending Callbacks → Idle/Prepare → Poll → Check → Close Callbacks.

- `process.nextTick()` > Promise microtasks > `setImmediate()` > `setTimeout(fn, 0)` in priority.
- Never run blocking synchronous code (heavy loops, `readFileSync`) inside request handlers — it freezes the whole server.
- Use try/catch with `async/await`, custom Error subclasses, and centralized Express error middleware.

Module 2 — Interview Questions

Q1. What's the difference between `let`, `const`, and `var`?

A: `var` is function-scoped and hoisted with an undefined initial value, which can leak variables outside blocks. `let` and `const` are block-scoped and exist in a 'temporal dead zone' until their declaration line, throwing a `ReferenceError` if accessed earlier. `const` additionally prevents reassigning the binding (though object/array contents it points to can still be mutated).

Q2. What is a closure? Give a practical backend example.

A: A closure is a function that retains access to variables from its enclosing scope even after that outer function has returned. A practical backend example is a rate-limiting middleware factory: `createRateLimiter(limit)` returns a middleware function that keeps a private 'count' variable alive across every request, without exposing it globally.

Q3. Why was `async/await` introduced if Promises already existed?

A: Promises solved callback hell but chaining many `.then()` calls was still verbose and harder to read, especially with conditional logic or loops. `async/await` is syntactic sugar over Promises that lets asynchronous code be written and read almost like synchronous code, including using ordinary try/catch for error handling.

Q4. Explain, step by step, why this prints 'A D C B': `console.log('A');` `setTimeout(() => console.log('B'), 0);` `Promise.resolve().then(() => console.log('C'));` `console.log('D');`

A: `console.log('A')` and `console.log('D')` run synchronously on the call stack first, printing A and D immediately. `setTimeout` schedules 'B' as a macrotask in the timer queue, while `.then()` schedules 'C' as a microtask. Once the stack is empty, the engine ALWAYS drains the microtask queue before any macrotask, so 'C' prints next, and finally the timer's macrotask runs, printing 'B' last.

Q5. Is JavaScript single-threaded? How does Node.js handle concurrency then?

A: Yes, JavaScript's main thread executes one operation at a time via the call stack. Node achieves concurrency by offloading I/O-bound work (file access, network calls, DNS) to libuv's event loop and a background thread pool, then queuing their callbacks to run on the main thread once the work completes — so the main thread is never blocked waiting.

Q6. What's the difference between `setImmediate()` and `setTimeout(fn, 0)`?

A: Both schedule a macrotask, but they run in different event loop phases — `setImmediate()` runs in the Check phase, while `setTimeout(fn, 0)` runs in the Timers phase. Their relative order is not guaranteed in the top-level module, but inside an I/O callback, `setImmediate()` will always run before a same-tick `setTimeout(fn, 0)`, since Check follows Poll directly.

Q7. What happens if you put a CPU-heavy synchronous loop inside an Express route handler?

A: It blocks the single main thread entirely until the loop finishes, meaning the event loop cannot process any other pending request, timer, or I/O callback during that time. Every other user hitting

the server experiences a stall, even on completely unrelated endpoints — this is why CPU-bound work should be offloaded to worker threads or a separate service.

Q8. What is the difference between `Promise.all` and `Promise.allSettled`?

A: `Promise.all` resolves with an array of values only if every promise fulfills, but rejects immediately as soon as any single promise rejects, discarding the results of the others. `Promise.allSettled` always waits for every promise to finish regardless of outcome, returning an array of `{status, value/reason}` objects, which is useful when you need to know the result of each operation independently.

Q9. How would you handle an uncaught error so it doesn't crash your whole Node.js process unexpectedly?

A: Wrap risky synchronous code in `try/catch`, use `async/await` with `try/catch` (or `.catch()` on promise chains) for asynchronous code, and add centralized Express error-handling middleware as a safety net for route handlers. As a last line of defense, listen for process-level `'uncaughtException'` and `'unhandledRejection'` events to log the error and gracefully shut down rather than leaving the process in an unknown state.

Q10. What is the microtask queue and why does it run before macrotasks?

A: The microtask queue holds callbacks from Promises (`.then/.catch/.finally`) and `queueMicrotask()`, and by the JavaScript specification it is always fully drained immediately after the currently executing script/task finishes, before the event loop is allowed to pick up the next macrotask (like a timer or I/O callback). This design guarantees that Promise-based code resolves as soon as possible, with the most predictable, consistent timing.

Module 3 — Node.js Complete

Node.js is the runtime that lets JavaScript run outside the browser, on a server. This module covers what Node actually is under the hood, how its concurrency model works, the most important built-in modules, and the package ecosystem (npm) that makes Node development so fast.

3.1 What Is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JavaScript to run outside the browser — on servers, in command-line tools, and in desktop apps. It was created in 2009 by Ryan Dahl specifically to solve the problem of handling many concurrent connections efficiently, using an event-driven, non-blocking I/O model rather than spinning up a thread per connection.

Node.js itself is not a language and not a framework — it's a runtime environment. It takes the V8 engine (which only understands JavaScript syntax and core built-ins like Array and Math) and bolts on capabilities a browser would never need: file system access, networking, process management, and more.

3.2 The V8 Engine

V8 is Google's open-source JavaScript engine, written in C++, originally built for the Chrome browser. It is the part of Node.js actually responsible for parsing and executing your JavaScript code.

What V8 does

- Parses JavaScript source code into an Abstract Syntax Tree (AST).
- Compiles JavaScript to machine code using Just-In-Time (JIT) compilation — rather than purely interpreting it line by line, which would be much slower.
- Manages the Heap (object storage) and the Call Stack (function execution tracking).
- Performs automatic garbage collection — reclaiming memory used by objects no longer reachable from your code.
- Continuously optimizes "hot" code paths that run frequently, recompiling them into faster machine code on the fly.

Key Insight

V8 alone gives you the JavaScript LANGUAGE — variables, functions, objects, Promises. It does NOT give you fs, http, or any networking capability. Those come from Node.js itself, layered on top of V8.

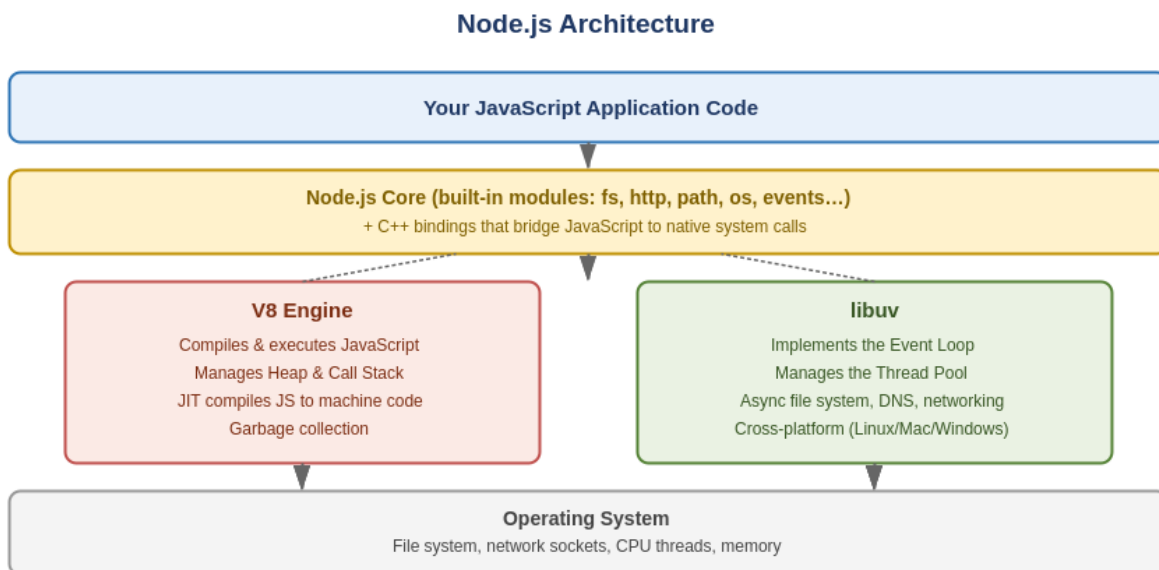


Figure 3.1 — How your code, Node's core modules, V8, libuv, and the operating system are layered together.

3.3 Event-Driven Architecture

Node.js is built around the idea that most things that happen — a request arriving, a file finishing reading, a timer expiring — are events. Instead of code blocking and waiting for these things to happen, Node registers a callback ("listener") to run automatically once the event occurs, and continues executing other code in the meantime.

This pattern is formalized in Node's EventEmitter class, which many core modules (and the HTTP server itself) are built on top of:

```
const EventEmitter = require('events');

class OrderService extends EventEmitter {
  placeOrder(order) {
    // ... save the order to the database ...
    this.emit('orderPlaced', order); // fire the event
  }
}

const orders = new OrderService();

orders.on('orderPlaced', (order) => {
  console.log(`Sending confirmation email for order ${order.id}`);
});

orders.on('orderPlaced', (order) => {
  console.log(`Notifying warehouse about order ${order.id}`);
});
```

```
orders.placeOrder({ id: 501 }); // both listeners fire, decoupled from placeOrder()
```

This is the same fundamental pattern used internally for HTTP requests (req and res are effectively event-driven streams), file system watches, and process signals.

3.4 The Single-Threaded Model

Node.js executes your JavaScript on a single main thread — there is only one call stack, and only one line of your application code runs at any given instant. This is a deliberate design choice, not a limitation that slipped through.

Why single-threaded works well for I/O-bound work

Traditional multi-threaded servers (e.g. classic Apache/PHP setups) spin up a new OS thread per connection. Threads are expensive — they consume memory (often 1-2MB of stack space each) and CPU time for context-switching. With thousands of concurrent connections, this model struggles to scale.

Node instead keeps one thread for your JS logic, and delegates the actually slow parts — disk reads, network calls, DNS lookups — to the operating system and a background thread pool (via libuv), picking the results back up via the event loop when they're ready. The result: Node can handle tens of thousands of concurrent connections on a single thread, because it's never sitting idle waiting — it's always either executing JS or has handed off waiting work elsewhere.

Important Nuance

Single-threaded applies to YOUR JavaScript code. Node itself uses multiple OS-level threads internally (the libuv thread pool, V8's garbage collector threads) — you just never see or control them directly from JS unless you explicitly use Worker Threads.

The cost: CPU-bound work is dangerous

Because there's only one thread, any synchronous, CPU-intensive task (image processing, heavy computation, large synchronous loops) blocks that single thread completely — freezing every other request your server is handling, even unrelated ones, until it finishes.

```
// This single request blocks ALL other users of your server
app.get('/fibonacci/:n', (req, res) => {
  function fib(n) { return n <= 1 ? n : fib(n - 1) + fib(n - 2); }
  res.json({ result: fib(Number(req.params.n)) }); // CPU-bound, synchronous
});
```

For genuinely CPU-bound work, Node provides the `worker_threads` module to run JavaScript on separate threads, or you offload the work to a separate process/service entirely.

3.5 Non-Blocking I/O

"Non-blocking I/O" means that when Node needs to perform an input/output operation (reading a file, querying a database, making a network request), it does not pause and wait for that operation to complete. Instead, it initiates the operation, registers a callback, and immediately moves on to execute other code. When the I/O operation eventually finishes, its callback is queued to run.

Blocking vs non-blocking, side by side

```
const fs = require('fs');

// BLOCKING -> the entire thread halts until the file is fully read
console.log('Before read');
const data = fs.readFileSync('large-file.txt', 'utf-8');
console.log('After read'); // only runs once reading is 100% finished
console.log('This had to WAIT, even though it has nothing to do with files');

// NON-BLOCKING -> reading happens in the background
console.log('Before read');
fs.readFile('large-file.txt', 'utf-8', (err, data) => {
  console.log('After read (inside callback)');
});
console.log('This runs IMMEDIATELY, before the file is even done reading');
```

Output order for the non-blocking version: 'Before read', 'This runs IMMEDIATELY...', then eventually 'After read (inside callback)' once the disk I/O actually completes.

Backend Implication

This is exactly why Node.js excels at I/O-heavy backend workloads (APIs, real-time apps, microservices that mostly wait on databases/network) but is a poor fit for CPU-heavy workloads (video encoding, heavy data crunching) compared to multi-threaded runtimes — unless you explicitly use worker threads or offload to another service.

3.6 Core (Built-in) Modules

Node ships with a standard library of built-in modules that require no installation — just `require()` (CommonJS) or `import` (ES Modules) them directly. These cover file systems, networking, OS info, and more.

fs — File System

Used to read, write, update, delete, and watch files and directories. Almost every method exists in three forms: synchronous (blocking, ...Sync suffix), callback-based (async, original style), and Promise-based (async, modern style via `fs.promises` or `require('fs/promises')`).

```
const fs = require('fs');
const fsPromises = require('fs/promises');

// 1. Synchronous -> BLOCKS the event loop until done (avoid in servers)
```

```
const data = fs.readFileSync('config.json', 'utf-8');

// 2. Callback-based -> non-blocking, classic Node style
fs.readFile('config.json', 'utf-8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// 3. Promise-based -> non-blocking, modern style, works with async/await
async function loadConfig() {
  const data = await fsPromises.readFile('config.json', 'utf-8');
  return JSON.parse(data);
}

// Writing and appending
await fsPromises.writeFile('output.txt', 'Hello World');
await fsPromises.appendFile('log.txt', 'New log entry\n');

// Directory operations
const files = await fsPromises.readdir('./uploads');
await fsPromises.mkdir('./temp', { recursive: true });
```

path — Path Manipulation

Provides utilities for working with file and directory paths in a way that's safe across different operating systems (Windows uses backslashes, Unix uses forward slashes).

```
const path = require('path');

path.join('/users', 'aanya', 'documents', 'file.txt');
// -> '/users/aanya/documents/file.txt' (OS-correct separators)

path.resolve('uploads', 'image.png');
// -> absolute path from the current working directory

path.basename('/uploads/photo.png'); // -> 'photo.png'
path.extname('/uploads/photo.png'); // -> '.png'
path.dirname('/uploads/photo.png'); // -> '/uploads'

path.parse('/uploads/photo.png');
// -> { root: '/', dir: '/uploads', base: 'photo.png', ext: '.png', name: 'photo' }
```

http — Building Servers and Making Requests

The raw http module is what frameworks like Express are built on top of. Understanding it helps you understand what's really happening underneath those frameworks.

```
const http = require('http');

const server = http.createServer((req, res) => {
  console.log(req.method, req.url);
```

```

if (req.url === '/' && req.method === 'GET') {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ message: 'Hello from raw Node.js!' }));
} else {
  res.writeHead(404, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Not found' }));
}
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});

```

Why frameworks exist

Notice how much manual work this is -- parsing URLs, checking methods, setting headers by hand. Express, Fastify, and similar frameworks exist to remove this boilerplate with routing, middleware, and body parsing built in.

os — Operating System Information

```

const os = require('os');

os.platform();      // 'linux', 'darwin', 'win32'
os.cpus().length;   // number of logical CPU cores
os.totalmem();      // total system memory in bytes
os.freemem();       // free system memory in bytes
os.hostname();      // machine's hostname
os.uptime();        // system uptime in seconds

```

Commonly used to determine how many worker processes to spawn (e.g. one per CPU core when using the cluster module for load balancing across cores).

events — The EventEmitter Class

Already introduced in section 3.3, this module provides the EventEmitter base class that powers Node's event-driven design — used directly, or inherited by many other core modules (streams, the http server, etc.).

```

const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('greet', (name) => console.log(`Hello, ${name}!`));
emitter.once('init', () => console.log('Runs only the first time'));

emitter.emit('greet', 'Dev'); // -> 'Hello, Dev!'
emitter.emit('init');         // -> 'Runs only the first time'
emitter.emit('init');         // -> nothing happens, 'once' already fired

emitter.off('greet', listenerFn); // remove a specific listener

```

3.7 npm (Node Package Manager)

npm is the default package manager for Node.js — it installs, manages, and publishes reusable packages of code ("modules") from the public npm registry, the largest software registry in the world.

Essential npm commands

Command	Purpose
npm init -y	Creates a new package.json with default values
npm install express	Installs a package and adds it to dependencies
npm install -D nodemon	Installs a package as a devDependency (dev-only tool)
npm install	Installs every dependency listed in package.json
npm uninstall express	Removes a package and its entry from package.json
npm update	Updates packages to the latest version allowed by semver ranges
npm run <script>	Runs a custom script defined in package.json's "scripts"
npm ci	Clean install strictly from package-lock.json — used in CI/CD pipelines
npx <package>	Runs a package's binary without installing it globally

Understanding semantic versioning (semver)

```
"express": "^4.18.2"
      |  |  |
      |  |  |
      MAJOR MINOR PATCH
```

Symbol / Term	Meaning
MAJOR	Breaking changes — code may need updates to keep working
MINOR	New backward-compatible features added
PATCH	Backward-compatible bug fixes only
^4.18.2	Allows MINOR and PATCH updates (4.x.x), but never 5.0.0
~4.18.2	Allows only PATCH updates (4.18.x)
4.18.2 (exact)	Locks to that exact version, no automatic updates

dependencies vs devDependencies

dependencies are required for the app to RUN in production (express, mongoose).

devDependencies are only needed during development (nodemon, jest, eslint) and are typically excluded from production installs with `npm install --omit=dev`.

3.8 package.json

package.json is the manifest file at the root of every Node.js project. It describes the project's metadata, dependencies, and the scripts used to run, build, and test it. It is the single most important file for understanding any Node.js codebase you open.

```
{
  "name": "ecommerce-api",
  "version": "1.2.0",
  "description": "Backend API for the e-commerce platform",
  "main": "src/index.js",
  "type": "module",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "test": "jest",
    "lint": "eslint ."
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.3",
    "dotenv": "^16.0.3"
  },
  "devDependencies": {
    "nodemon": "^2.0.22",
    "jest": "^29.5.0"
  },
  "engines": {
    "node": ">=18.0.0"
  }
}
```

Key fields explained

Field	Purpose
name / version	Package identity, following semver
main	Entry point file when this package is require()'d by another project
type	"module" enables ES Modules (import/export); default/"commonjs" uses require()
scripts	Named shortcuts run via <code>npm run <name></code> (e.g. <code>npm run dev</code>)

Field	Purpose
dependencies	Packages required in production
devDependencies	Packages only needed for local development/testing
engines	Specifies which Node.js versions the project supports

package-lock.json (auto-generated, should be committed to version control) records the EXACT resolved version of every package and sub-dependency, guaranteeing that every developer and every deployment installs an identical dependency tree.

Module 3 — Revision Notes (Quick Recall)

- Node.js = a runtime that lets JS run outside the browser, built on V8 + libuv.
- V8 handles parsing/compiling/executing JS, the heap, the call stack, and garbage collection — nothing networking-related.
- Node is event-driven: EventEmitter is the core pattern behind many built-in modules.
- Node's JS execution is single-threaded; I/O work is delegated to libuv's thread pool / the OS.
- Non-blocking I/O: operations are initiated, callback registered, execution continues immediately.
- fs has sync, callback, and Promise-based APIs — prefer Promise-based (fs/promises) in servers.
- path normalizes file paths across operating systems (join, resolve, basename, extname).
- http is the low-level module that frameworks like Express are built on top of.
- os exposes system info (CPU count, memory) often used to size worker pools.
- npm manages packages; semver (MAJOR.MINOR.PATCH) and ^ / ~ control update ranges.
- package.json declares metadata, scripts, and dependencies; package-lock.json pins exact versions.

Module 3 — Interview Questions

Q1. What exactly is Node.js, and what is it NOT?

A: Node.js is a JavaScript runtime environment that executes JS outside the browser, built on Google's V8 engine plus the libuv library for async I/O and the event loop. It is not a programming language (it runs JavaScript) and not a web framework (Express, Fastify, etc. are built on top of it, not part of it).

Q2. What is the relationship between Node.js, V8, and libuv?

A: V8 is the engine that parses and executes the JavaScript language itself and manages memory (heap, garbage collection). libuv is the C library that implements the event loop, the thread pool, and cross-platform asynchronous I/O. Node.js is the runtime that combines V8 with libuv and adds JavaScript-callable APIs (fs, http, os, etc.) bridging to the operating system.

Q3. Why is Node.js single-threaded, and is that a weakness?

A: It's a deliberate design to avoid the memory and context-switching overhead of spawning a thread per connection, which makes it extremely efficient for I/O-bound workloads with high concurrency. It becomes a weakness only for CPU-bound work, since a single heavy synchronous computation blocks the only thread and stalls every other request — that's mitigated with `worker_threads` or offloading to separate services.

Q4. Explain non-blocking I/O with a real example.

A: Non-blocking I/O means Node starts an operation like a file read or DB query and, instead of waiting, immediately continues executing the rest of the synchronous code, registering a callback to run once the operation completes. For example, `fs.readFile()` returns control to the next line of code instantly, and the file's contents only get processed inside its callback once the disk operation actually finishes in the background.

Q5. What's the difference between `fs.readFileSync`, `fs.readFile`, and `fs.promises.readFile`?

A: `fs.readFileSync` is synchronous and blocking — it halts the entire thread until the file is read, which is dangerous in a server handling concurrent requests. `fs.readFile` is asynchronous and callback-based, the original non-blocking style. `fs.promises.readFile` is also asynchronous and non-blocking, but returns a Promise instead of taking a callback, making it compatible with `async/await` for cleaner code.

Q6. What problem does the `path` module solve?

A: It abstracts away operating-system differences in file path formatting — Windows uses backslashes while Unix-based systems use forward slashes — so code that builds or parses paths works correctly everywhere. It also provides convenient utilities like extracting a file's extension, directory, or base name without manual string manipulation.

Q7. What is semantic versioning, and what does `^4.18.2` mean in `package.json`?

A: Semantic versioning is a MAJOR.MINOR.PATCH numbering scheme where MAJOR signals breaking changes, MINOR signals new backward-compatible features, and PATCH signals backward-compatible bug fixes. The caret (`^4.18.2`) tells npm it's safe to automatically install newer MINOR and PATCH versions (anything in the 4.x.x range), but never auto-upgrade to a new MAJOR version like 5.0.0, since that could break the app.

Q8. What's the difference between `dependencies` and `devDependencies` in `package.json`?

A: `dependencies` lists packages required for the application to actually run in production, like `express` or a database driver. `devDependencies` lists packages only needed during development or testing, like `nodemon`, `eslint`, or a testing framework, and these are typically skipped when installing for a production deployment.

Q9. Why should `package-lock.json` be committed to version control?

A: It records the exact resolved version of every direct and transitive dependency that was installed, down to the exact build. Without it, two different developers (or a developer's machine vs. a CI/CD pipeline) could install slightly different dependency versions allowed by the semver ranges in `package.json`, leading to inconsistent behavior or 'works on my machine' bugs.

Q10. How does Node.js's `EventEmitter` relate to its overall event-driven architecture?

A: `EventEmitter` is the foundational class that implements the publish/subscribe pattern Node relies on throughout its core: code emits named events with `emit()`, and any number of listener functions registered via `on()` get called when that event fires. Core modules like the HTTP server and streams

are built directly on top of EventEmitter, which is why understanding it explains how req/res objects and many async APIs in Node actually behave internally.